

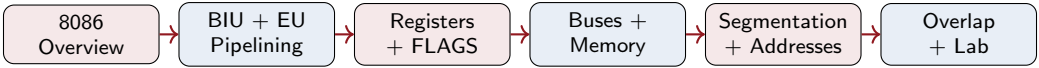
PRESIDENCY UNIVERSITY

CSE322: Microprocessors Based System Design Laboratory

Lecture 03: 8086 Architecture, Registers & Memory Organization

Afzal Hossan
Lecturer
Department of Computer Science and Engineering
Presidency University

Lecture Roadmap



Teaching Sequence

We first understand the processor, then its registers and buses, then memory organization, and finally use segmentation to calculate real physical addresses.

Learning Goals

At the end of this lecture, students will be able to:

1. Explain the main features and CISC nature of Intel 8086.
2. Distinguish the internal architecture from the external bus interface.
3. Explain the Bus Interface Unit (BIU) and Execution Unit (EU).
4. Explain the 6-byte instruction queue and how BIU/EU overlap fetch and execution.
5. Identify all major 8086 registers and state their basic functions.
6. Explain the address, data and control buses and relate bus width to memory capacity.
7. Explain the important external signals, operating modes and basic bus-cycle timing.
8. Explain byte-addressable memory, even/odd banks, A0, BHE and little-endian storage.
9. Explain memory segmentation, segment size, segment boundaries and segment overlap.
10. Calculate segment bases, physical addresses, segment ranges and boundary cases.
11. Explain the 1 MB physical address space and the original 8086 wrap-around case.
12. Trace basic instruction fetching and memory organization using architecture diagrams.

8086 Overview and Key Specifications

What is 8086?

Intel 8086 is a 16-bit CISC microprocessor. Its ALU, general-purpose registers and data bus are 16 bits wide, while its address bus is 20 bits wide.

- Introduced in 1978.
- 16-bit word processing.
- 20-bit physical addressing.
- Byte-addressable memory.
- 6-byte instruction prefetch queue.

Feature	Value
ALU / word size	16 bits
General registers	16 bits
Data bus	16 bits
Address bus	20 bits
Physical memory	2^{20} bytes = 1 MB
Offset	16 bits = 64 KB window
Prefetch queue	6 bytes

Key idea

16-bit processing width does not limit the physical address to 16 bits. The 20-bit address bus lets 8086 select 1 MB of byte-addressable memory.

Why Is 8086 Called a CISC Processor?

CISC = Complex Instruction Set Computer

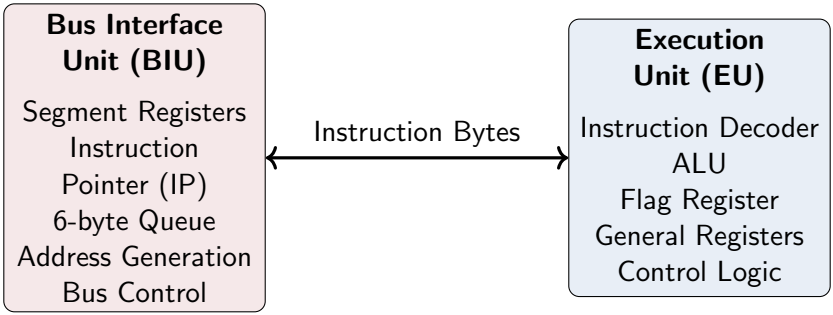
8086 supports a rich instruction set, variable-length instructions and instructions that can operate on registers or memory operands.

- Instructions can be different lengths.
- Many instructions can operate on memory operands.
- A single instruction may perform several low-level operations.

Why it matters in this lecture

The variable-length instruction stream is one reason the BIU prefetch queue and instruction-fetch mechanism are important when understanding 8086 performance.

8086 Internal Architecture: Complete View



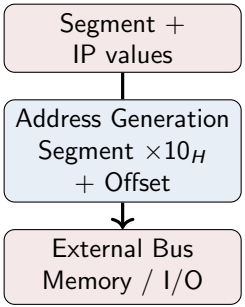
8086 Internal Design Philosophy

The 8086 is divided into two independent units so that fetching and execution can happen simultaneously.

Bus Interface Unit (BIU): What Does It Do?

Main BIU resources

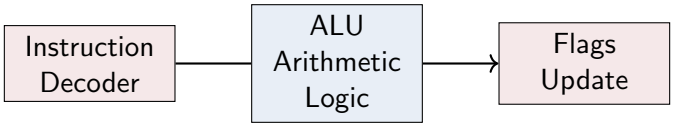
- CS, DS, SS and ES segment registers
- Instruction Pointer (IP)
- 6-byte instruction prefetch queue
- Address-generation circuitry
- Bus-control and external-bus interface logic



BIU's central job

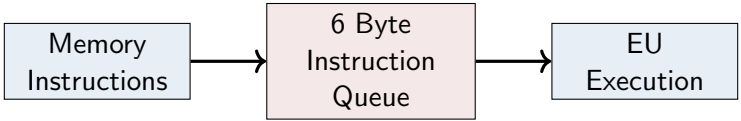
The BIU connects the 8086 core to memory/I/O, forms physical addresses and fetches instruction bytes ahead of execution.

Internal Architecture: Role of EU Components



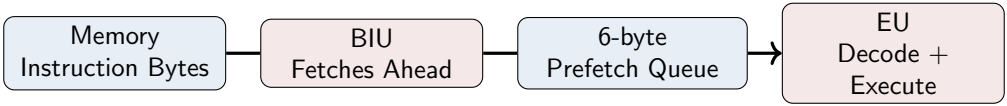
- Decoder understands the instruction opcode.
- ALU performs arithmetic and logical operations.
- Flag register stores result conditions.
- Control logic coordinates all EU operations.

Internal Architecture: Instruction Queue



- BIU continuously fetches instruction bytes.
- Queue stores upcoming instructions temporarily.
- EU removes instructions from queue for execution.
- This reduces CPU waiting time.

Instruction Pipelining in 8086: BIU + EU Work in Parallel



- EU normally executes one instruction at a time.
- While EU executes the current instruction, BIU can fetch future instruction bytes.
- The queue buffers prefetched bytes between BIU and EU.

Important

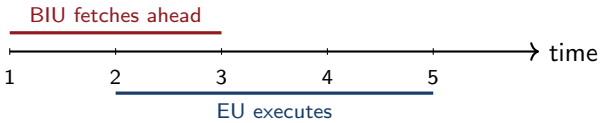
Pipelining does **not** mean that 8086 executes two instructions simultaneously. It overlaps **fetching** with **execution**.

Benefit

Less idle time between instructions and better use of the bus and execution unit when sequential code is being executed.

Instruction Pipelining: Conceptual Timing

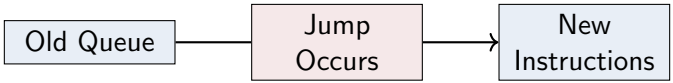
Pipeline step	BIU	EU
1	Fetch instruction 1	Prepare / idle
2	Fetch instruction 2	Execute instruction 1
3	Fetch instruction 3	Execute instruction 2
4	Fetch instruction 4	Execute instruction 3
5	Fetch instruction 5	Execute instruction 4



Conceptual timing: actual instructions require different numbers of clock cycles; the key idea is overlap between BIU fetching and EU execution.

Pipeline Limitations in 8086

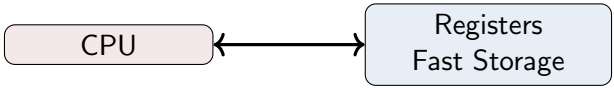
- **Jump Instructions:** The queue may contain wrong instructions after a branch.
- **Interrupts:** Current queue contents may need to be discarded.
- **Memory Delay:** Slow memory access can reduce pipeline efficiency.



What is a Register?

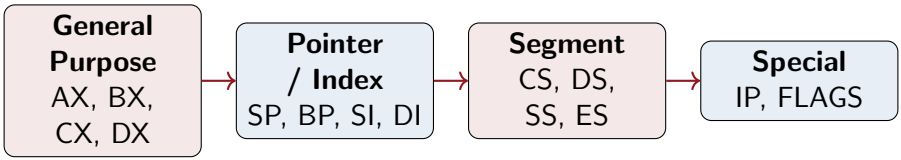
Definition

A register is a small, high-speed storage location inside the CPU used to temporarily hold data, instructions, addresses and intermediate results during processing.



- Registers are faster than RAM.
- 8086 contains several 16-bit registers.
- Different registers have different purposes.

Complete 8086 Register Set



- The 8086 programming model exposes 14 principal 16-bit registers: AX, BX, CX, DX, SP, BP, SI, DI, CS, DS, SS, ES, IP and FLAGS.
- Only AX, BX, CX and DX have separately addressable 8-bit halves.
- SP/BP are stack-related pointer registers; SI/DI are index registers. Detailed stack operations and advanced addressing are covered later.
- IP identifies the next instruction offset; FLAGS records status and control information.

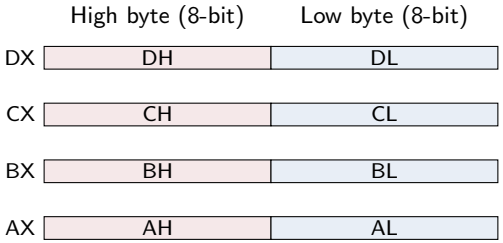
Pointer and Index Registers: Basic Roles

Register	Role	Architecture connection
SP	Stack Pointer	Offset associated with the current stack top
BP	Base Pointer	Pointer used with stack-based data
SI	Source Index	Index register for data locations
DI	Destination Index	Index register for data locations

Scope

These registers are identified here because they are part of the 8086 programming model. Their detailed addressing-mode use is taught in the next instruction-set/addressing lecture.

General-Purpose Registers: AX, BX, CX and DX

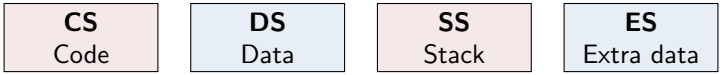


- **AX** — accumulator; frequently used by arithmetic and data-processing instructions.
- **BX** — base register; often useful when working with memory addresses.
- **CX** — count register; commonly used for counts and repetition.
- **DX** — data register; used with multiplication/division and some I/O operations.

Important

Only AX, BX, CX and DX can be split into two independently addressable 8-bit halves: AH/AL, BH/BL, CH/CL and DH/DL.

Segment Registers: Four Segment Windows

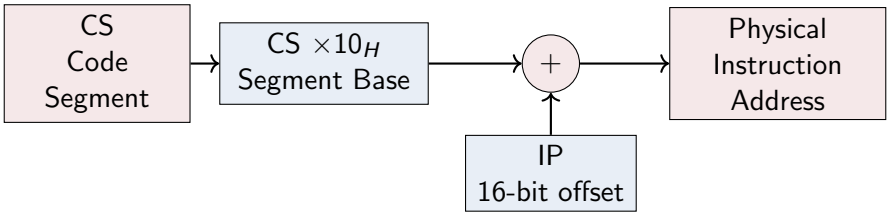


Register	Segment role	Common architectural pair
CS	Code segment	CS:IP for instruction fetch
DS	Data segment	Default segment for many data references
SS	Stack segment	Stack-related offsets such as SP/BP
ES	Extra segment	Additional data segment

Architecture idea

Each segment register stores a 16-bit segment value, not a full physical address. The BIU combines that value with a 16-bit offset to generate the physical address.

Instruction Pointer (IP): Where Does Execution Continue?



Meaning

IP is a 16-bit offset pointing to the next instruction within the current code segment. The BIU uses CS:IP to form the instruction's physical address.

$$\text{Instruction physical address} = CS \times 10_H + IP$$

FLAGS Register: Status and Control Information

The 8086 FLAGS register is 16 bits wide. The important defined bits are not consecutive; their actual bit positions matter.

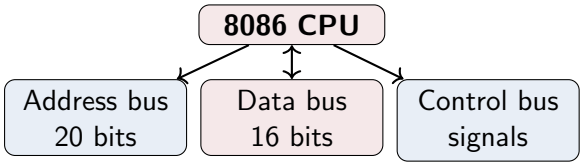
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	R	R	R	R	OF	DF	IF	TF	SF	ZF	R	AF	R	PF	R	CF

- **CF (0):** carry / borrow
- **PF (2):** parity of low byte
- **AF (4):** auxiliary carry
- **ZF (6):** result is zero
- **SF (7):** sign of signed result
- **TF (8):** single-step trap
- **IF (9):** interrupt enable
- **DF (10):** string direction
- **OF (11):** signed overflow

Exam Tip

CF and OF are different: CF describes an unsigned carry/borrow condition; OF describes signed arithmetic overflow.

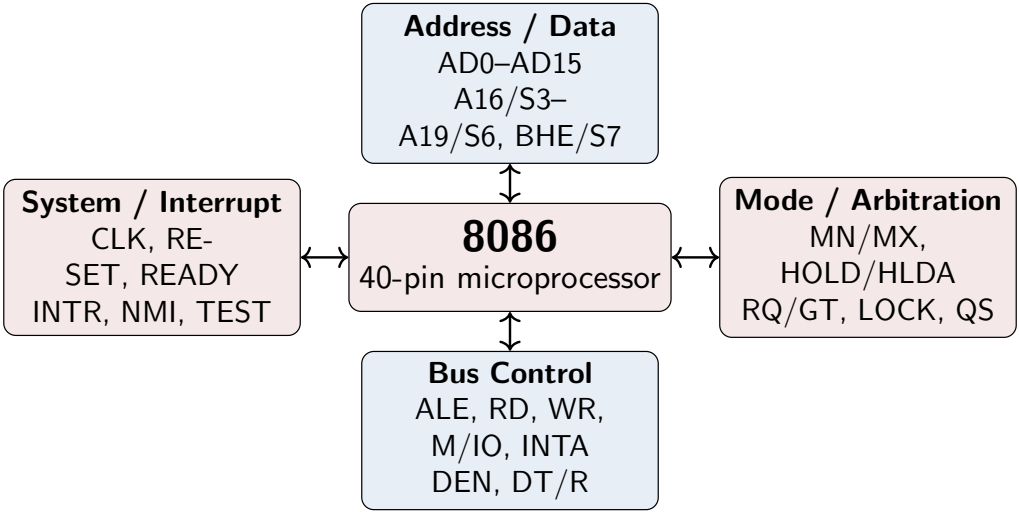
8086 Buses and Memory Capacity



- **Address bus:** selects a physical memory location.
- **Data bus:** carries data between CPU and memory/I/O.
- **Control bus:** carries signals such as read, write, interrupt and bus-control information.

20-bit address bus: 2^{20} bytes $= 1$ MB.

8086 External Architecture: 40-Pin Interface



8086 External Signals: Important Groups

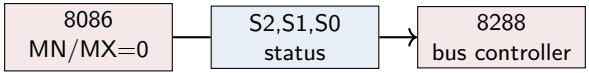
Signal group	Function
AD0–AD15	Lower address during T1; data during later bus states
A16/S3–A19/S6	Upper address during T1; status later
BHE/S7	Enables the high/odd byte bank when active
ALE	Tells the external latch to capture the multiplexed lower address
RD / WR	Control external read and write transfers
M/IO, INTA	Memory/I/O and interrupt acknowledge control
READY	Allows insertion of wait states for slow memory/I/O
MN/MX	Selects minimum or maximum operating mode
HOLD/HLDA, RQ/GT	Request/grant bus ownership
LOCK, QS	Bus-lock and instruction-queue status functions

Focus

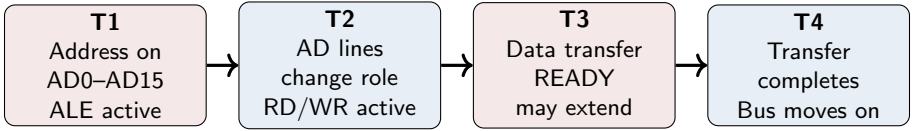
Understand what each signal group does; exact pin numbering is secondary in Lecture 03.

8086 Minimum Mode vs Maximum Mode

Mode	Who generates bus controls?	Main idea
Minimum (MN/MX=1)	8086 directly	Simpler single-processor system
Maximum (MN/MX=0)	8288 bus controller from 8086 status	Supports more complex system configurations



8086 Bus Cycle: T1–T4 and Multiplexed Address/Data



1. **T1:** lower address appears on AD0–AD15; ALE tells the external latch to capture it.
2. **T2:** AD0–AD15 are reused for data; read/write control begins.
3. **T3:** data is transferred; a slow device may keep READY low.
4. **T4:** the bus transfer finishes.

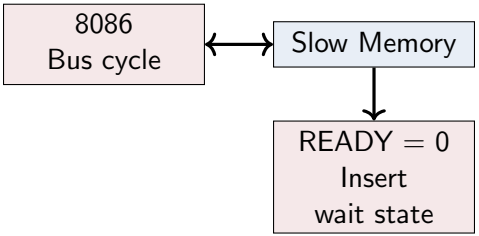
Why multiplex address and data?

The same pins are reused to reduce the number of package pins. The address is latched during T1, leaving AD0–AD15 available for data during later states.

Read vs Write

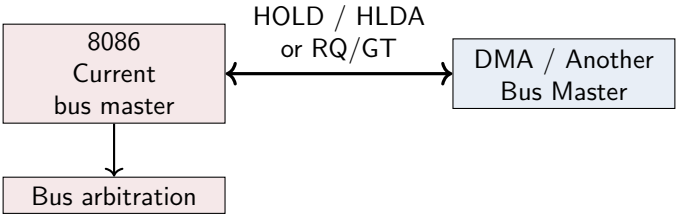
Read: memory $\rightarrow$ 8086. Write: 8086 $\rightarrow$ memory.

READY and Wait States: What If Memory Is Slow?



- The normal cycle uses T1–T4.
- If the external device is not ready, it can hold **READY low**.
- The processor inserts one or more **Tw (wait) states** before completing the transfer.
- This allows a slower memory or peripheral to work with the faster CPU bus timing.

Bus Ownership: HOLD / HLDA and RQ/GT

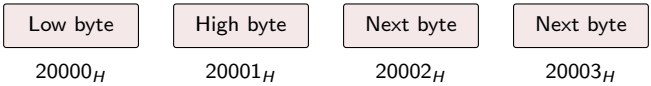


- In minimum mode, **HOLD** requests the bus and **HLDA** acknowledges that the 8086 has released it.
- In maximum mode, **RQ/GT** lines provide request/grant bus arbitration.
- The key idea is that the CPU may temporarily give another bus master access to the system bus.

Memory Is Byte-Addressable

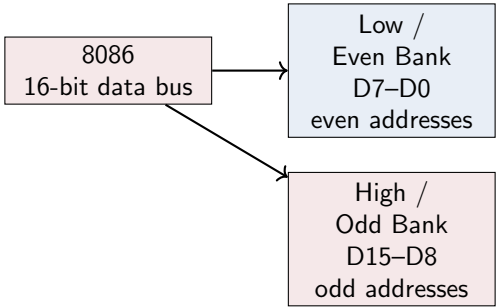
One address identifies one byte

In the 8086 memory space, each physical address refers to one 8-bit byte. A 16-bit word occupies two consecutive byte addresses.



- Byte addresses increase by 1.
- A word is two bytes, so word alignment matters for memory efficiency.
- The 20-bit address bus represents 2^{20} **byte locations**, not 2^{20} words.

Even and Odd Memory Banks in 8086



- The 1-MB byte-addressable memory is organized as two 8-bit banks to support the 16-bit data bus.
- **A0**=0 selects an even-address byte; **A0**=1 identifies an odd-address byte.
- $\overline{\mathrm{BHE}}$ enables the high-byte bank when asserted low.
- An even-aligned word can use both banks in one bus cycle; an odd-aligned word needs two bus cycles.

A0 and BHE: How Are the Memory Banks Selected?

$A0$	$\overline{\mathrm{BHE}}$	Transfer selected
0	1	Low/even bank: one byte at an even address
1	0	High/odd bank: one byte at an odd address
0	0	Both banks: one 16-bit word at an even address
1	1	Neither bank for a normal memory transfer

Odd-address word

For a 16-bit word starting at an odd address, the first cycle transfers the odd byte and the next cycle transfers the following even-address byte. This is why an unaligned word takes two bus cycles.

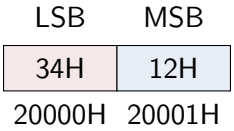
Little-Endian Storage in 8086

Rule

For a multi-byte word, the **least significant byte (LSB)** is stored at the lower memory address.

Suppose:

$$AX = 1234_H$$

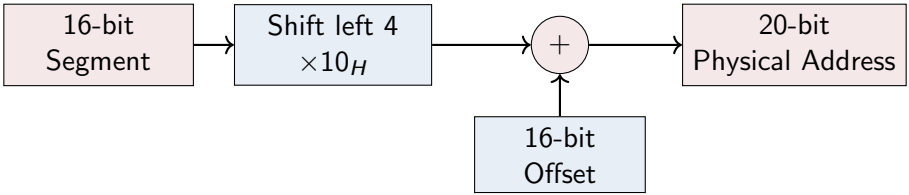


Lower address $\rightarrow$ lower-order byte

Why Segmentation?

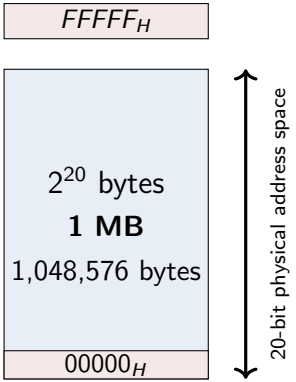
The Core Problem

8086 has 16-bit internal registers and a 16-bit offset, but its address bus is 20 bits wide. A single 16-bit register cannot directly represent every address from 00000_H to $FFFFF_H$.



$$\text{Physical Address} = (\text{Segment} \times 10_H) + \text{Offset}$$

Physical Memory: How Much Can 8086 Address?



- Lowest physical address: 00000_H.
- Highest physical address: $FFFFF_H$.
- Total physical addressable memory: $2^{20} = 1,048,576$ bytes $= 1024$ KB $= 1$ MB.

How Many Segments Are Possible?

Possible Segment Values

A segment register is 16 bits, so it can contain:

$$2^{16} = 65,536$$

values:

$$0000_H \text{ to } FFFF_H$$

Each value defines a base on a 16-byte boundary.

Active Segments at One Time

The 8086 has four segment registers:

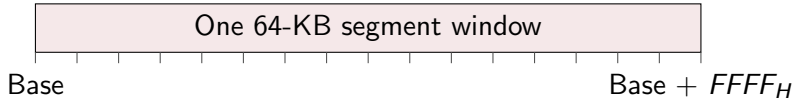
- CS – Code
- DS – Data
- SS – Stack
- ES – Extra

So four logical segment windows are selected for normal CPU operation at a time.

Important Distinction

65,536 possible segment starting values does not mean 65,536 independent non-overlapping 64-KB memory regions. The windows overlap heavily.

Segment Size and Starting-Address Granularity



- Offset is 16 bits: 0000_H through $FFFF_H = 65{,}536$ byte positions $= 64$ KB.
- A full segment window therefore covers 64 KB.
- Segment bases occur every 16 bytes (one **paragraph**), because the base is Segment $\times\ 10_H$.
- The hardware does not enforce that a program must use all 64 KB; a program may use a smaller portion of a segment.

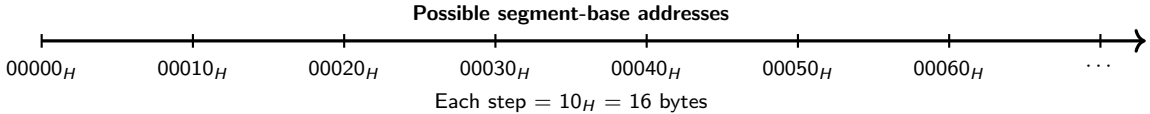
$$\boxed{\text{Segment Base} = \text{Segment Value} \times 10_H}$$

Where Can an 8086 Segment Start?

A segment does not start at an arbitrary byte address. The segment value is 16 bits, and the 8086 forms the segment base by shifting it left by four bits:

$$\boxed{\text{Segment Base} = \text{Segment Value} \times 10_H}$$

Therefore, every segment starts on a **16-byte boundary** (a paragraph boundary).



Examples

Segment value	Calculation	Segment base
0000_H	$0000_H \times 10_H$	00000_H
0001_H	$0001_H \times 10_H$	00010_H
0002_H	$0002_H \times 10_H$	00020_H
0010_H	$0010_H \times 10_H$	00100_H
$FFFF_H$	$FFFF_H \times 10_H$	$FFFF0_H$

Consecutive Segment Addresses: What Is the Difference?

The Rule

Consecutive segment values differ by **1**:

$$S_2 = S_1 + 1$$

Since the segment base is $S \times 10_H$:

$$(S_1 + 1) \times 10_H - S_1 \times 10_H = 10_H$$

$$\boxed{10_H = 16 \text{ bytes}}$$

Why this matters

A segment can cover up to 64 KB, but the next segment may begin only 16 bytes later. Therefore, consecutive segment windows overlap heavily.

Example of consecutive starts

Segment	Base	Change
1000_H	10000_H	–
1001_H	10010_H	$+10_H$
1002_H	10020_H	$+10_H$

Every new segment start = previous start + 16 bytes

Segment Range, Complete 64-KB Window and Boundary

For segment value S:

$$\text{Base} = S \times 10_H, \qquad \text{First} = S \times 10_H, \qquad \text{Last} = S \times 10_H + FFFF_H.$$

Example: $S = 3000_H$

$$\text{Base} = 30000_H$$

$$\text{Range} = 30000_H\text{–}3FFFF_H$$

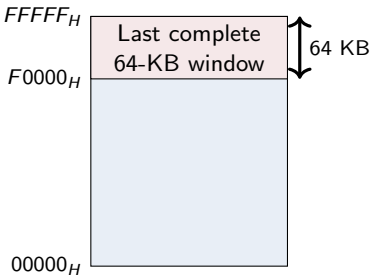
$$\text{Size} = 10000_H = 65,536 \text{ bytes} = 64 \text{ KB}$$

For a complete 64-KB window inside 1 MB:

$$\text{Base} + FFFF_H \leq FFFFF_H$$

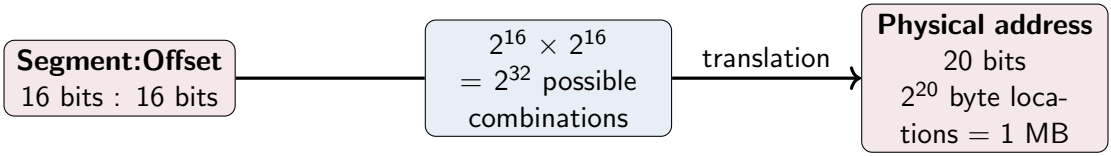
$$\boxed{\text{Base} \leq F0000_H}$$

$\boxed{F000_H \text{ is the last complete segment value}}$



Number of segment values that give a complete window: $F000_H + 1 = 61,441$.

Logical Address vs Physical Address

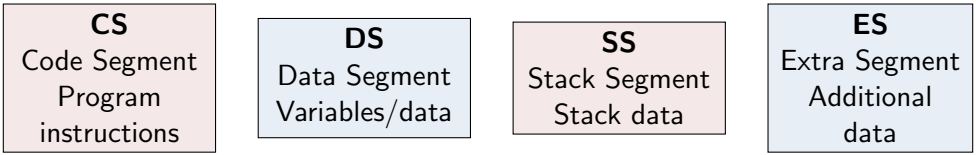


Do not confuse the counts

There are 2^{32} possible **Segment:Offset combinations**, not 4 GB of physical RAM. The 8086 has only 2^{20} unique physical byte addresses, so many logical addresses can refer to the same physical byte.

$$\text{Physical Address} = (\text{Segment} \times 10_H) + \text{Offset}$$

8086 Memory Segments: Types and Roles



- CS works with IP to fetch instructions: CS:IP.
- SS works with SP/BP for stack access: typically SS:SP or SS:BP.
- DS is the default segment for many data accesses.
- ES is especially useful as an additional data segment and as a destination segment for string operations.

Last Complete Segment vs Largest Segment Value

Meaning	Value	Result
Last physical byte	$FFFFF_H$	End of 1-MB physical memory
Last complete segment base	$F0000_H$	$F000_H : 0000_H$ through $F000_H : FFFF_H$
Largest segment value	$FFFF_H$	Base $FFFF0_H$; full window crosses 1 MB

Largest segment value calculation

$$FFFF_H \times 10_H = FFFF0_H$$

$$FFFF_H : FFFF_H = FFFF0_H + FFFF_H = 10FFEF_H$$

Original 8086 boundary behavior

Only 20 physical address bits are output. Therefore, the nominal carry beyond $FFFFF_H$ is not represented on the physical address bus; the address wraps modulo 2^{20}.

1-MB Wrap-Around: A Worked Boundary Example

For:

$$FFFF_H : 0010_H$$

$$FFFF0_H + 0010_H = 100000_H$$

The nominal result is one byte beyond the 20-bit range.

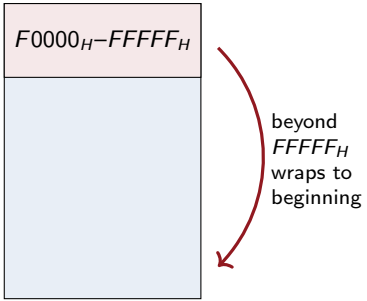
$$100000_H \bmod 100000_H = 00000_H$$

Thus, on the original 8086:

$$\boxed{FFFF_H : 0010_H \rightarrow 00000_H}$$

For the maximum offset:

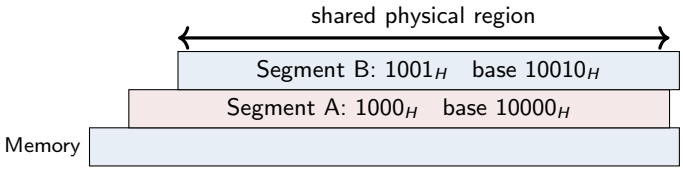
$$FFFF_H : FFFF_H \rightarrow 0FFEF_H$$



Important distinction

This is a boundary property of the original 8086's 20-bit physical address bus. Do not treat the nominal 21-bit arithmetic result as an additional physical address bit.

Segment Overlap: Why Different Logical Addresses Can Match



- Each segment window can cover 64 KB.
- Consecutive segment bases are only 16 bytes apart.
- Therefore, consecutive 64-KB windows overlap by 64 KB $-$ 16 bytes $= 65,520$ bytes.
- A physical byte can have multiple valid Segment:Offset representations.

Example

$$0000_H : 0010_H = 00010_H = 0001_H : 0000_H$$

Segment Overlap: Calculation and General Condition

Consider:

$$A = 1000_H : 0020_H, \qquad B = 1002_H : 0000_H$$

$$1000_H \times 10_H = 10000_H \qquad 1002_H \times 10_H = 10020_H$$

$$10000_H + 0020_H = 10020_H \qquad 10020_H + 0000_H = 10020_H$$

$$\boxed{1000_H : 0020_H = 1002_H : 0000_H = 10020_H}$$

General condition

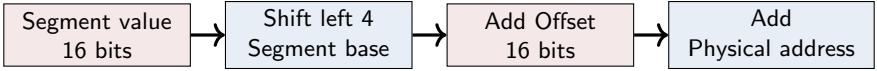
Two Segment:Offset pairs refer to the same physical byte when:

$$S_1 \times 10_H + O_1 = S_2 \times 10_H + O_2$$

Equivalently:

$$\boxed{(S_2 - S_1) \times 10_H = O_1 - O_2}$$

Physical Address Calculation: Complete Procedure



$$\boxed{PA = (S \times 10_H) + O}$$

Worked example

$$S = 1234_H, \qquad O = 5678_H$$

$$1234_H \times 10_H = 12340_H$$

$$12340_H + 5678_H = \boxed{179B8_H}$$

1. Write the segment value.
2. Append one hexadecimal zero (shift left 4 bits) to get the base.
3. Add the 16-bit offset.
4. If the result exceeds $FFFFF_H$, apply the original 8086 20-bit wrap-around rule.

Register analysis

Students should be able to:

- identify every major 8086 register,
- state the purpose of each register group,
- identify 16-bit registers and the 8-bit halves of AX/BX/CX/DX,
- identify CS, DS, SS, ES and IP in an architecture diagram.

Memory analysis

Students should be able to:

- calculate segment bases,
- calculate physical addresses,
- find segment ranges,
- determine whether two logical addresses overlap,
- explain the 1-MB boundary case.

Experiment

Study of 8086 architecture and physical address calculation.

Integrated Practice: Architecture + Segmentation

Given:

$$CS = 2400_H, \quad IP = 0150_H, \quad DS = 3500_H, \quad BX = 0200_H$$

1. **Instruction physical address:**

$$2400_H \times 10_H + 0150_H = \boxed{24150_H}$$

2. **Data physical address using DS and offset 0200_H:**

$$3500_H \times 10_H + 0200_H = \boxed{35200_H}$$

3. **DS segment range:**

$$35000_H \text{ to } 44FFF_H$$

4. **Question:** Find another Segment:Offset pair for 24150_H.

Hint

Increase the segment value by one and reduce the offset by 10_H, as long as the resulting segment and offset remain valid.

Integrated Challenge: Find an Alias

Suppose a program accesses physical address:

$$\boxed{10020_H}$$

One representation is:

$$1000_H : 0020_H$$

Can we find another representation?

Method

Increase the segment by one and reduce the offset by 10_H:

$$1001_H : 0010_H$$

$$1001_H \times 10_H + 0010_H = 10010_H + 0010_H = 10020_H$$

$$\boxed{1000_H : 0020_H = 1001_H : 0010_H = 1002_H : 0000_H = 10020_H}$$

Lecture 03 Practical Checklist

Before leaving the laboratory, students should be able to demonstrate:

- Draw and explain the BIU and EU.
- Explain the 6-byte instruction queue and pipelining.
- Identify all major 8086 registers.
- Explain 20-bit addressing and 1 MB memory.
- Explain the three bus types and bus widths.
- Explain AD0–AD15 multiplexing and ALE.
- Distinguish minimum and maximum mode.
- Explain T1–T4 bus-cycle states and READY wait states.
- Explain byte banks, A0, BHE and little endian.
- Calculate segment bases and physical addresses.
- Determine segment ranges, boundary cases and overlaps.
- Explain the original 8086 1-MB wrap-around.

Experiment: 8086 register analysis and physical address calculation practice.

Lecture 03 Summary: The Complete Mental Model

- 8086 is a 16-bit CISC microprocessor with a 20-bit address bus and 16-bit data bus.
- The internal architecture is divided into BIU and EU; the BIU fetches and the EU executes.
- The 6-byte prefetch queue lets instruction fetching overlap execution, although the EU still executes one instruction at a time.
- The register set includes AX/BX/CX/DX, SP/BP/SI/DI, CS/DS/SS/ES, IP and FLAGS.
- The external interface uses multiplexed address/data lines, ALE, bus-control signals and minimum/maximum operating modes.
- A normal bus transfer uses T1–T4; READY can add wait states.
- Memory is byte-addressable and organized into even/low and odd/high byte banks for the 16-bit data bus.
- The 8086 has $2^{20} = 1$ MB of physical byte-addressable memory, from 00000_H to $FFFFF_H$.
- A 16-bit segment value produces a base on a 16-byte boundary; the offset provides up to 64 KB within the segment window.
- There are 65,536 possible segment values and 65,536 possible segment bases, spaced 16 bytes apart.
- A complete 64-KB window must start at or below $F0000_H$; therefore $F000_H$ is the last complete segment value.
- Physical address is $PA = S \times 10_H + O$; different Segment:Offset pairs can map to the same physical byte.
- $FFFF_H$ is the largest segment value, but its full window crosses the 1-MB boundary; on the original 8086 the nominal address wraps modulo 2^{20}.

Final takeaway

Architecture explains how 8086 moves, organizes and addresses data.
The next lecture can build on this hardware foundation without repeating it.